Lab 08 – Worksheet

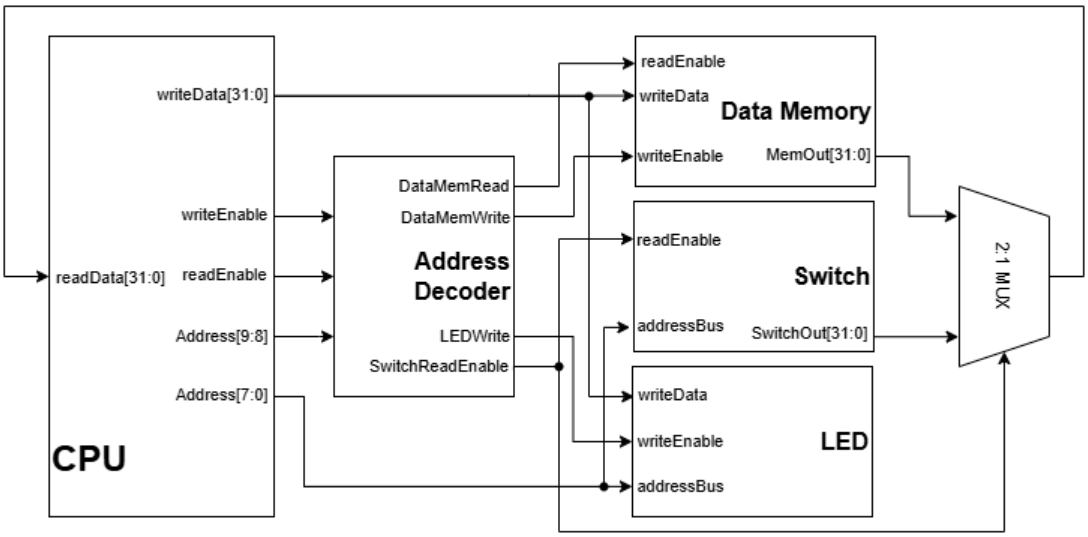
Name: Abbas Huzaifa Dar, Syed Muhammad Kabir	ID: ad11013, sk11236	Section: T4

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagram
a) Data Memory Module
b) Address Decoder
c) Top Module

All modules implemented in Top:



Comments

All of the modules have altogether been implemented in the top module. The CPU gives the signals to the address signal upon which it generates signals for Data Memory, Switch and LED blocks. The 32 bit writeData signal is fed into both the Data Memory and LED inputs from the CPU. The lower 8 bits of address are fed into the Switch and LED modules to get desired 32 bit output, which gets fed again into the CPU into its readData input.

Task 2

*Modules for **DataMemory, AddressDecoder, AddressDecoderTop, Switches and Leds.***

DataMemory:

```
module DataMemory(
    input clk,
    input rst,
    input MemWrite,
    input [31:0] address,
    input [31:0] write_data,
    output reg [31:0] read_data
   );
   reg [31:0] mem[0:511];
   always @(posedge clk or posedge rst) begin
      if(rst) begin
         read_data <= 32'b0;
      end
      else if(MemWrite == 1'b1) begin
         mem[address[8:0]] <= write_data;
      end
      else begin
         read_data <= mem[address[8:0]];
      end
   end
endmodule
```

AddressDecode:

```
module Address_Decoder(
input clk,
input rst,
input [31:0] address,
input readEnable, writeEnable, writeLed,
input [31:0] writeData,
input [15:0] switches,
input [31:0] datamemory,
output memWrite,
output reg [31:0] readData,
output reg [15:0] leds
);
wire [1:0] selector;
assign selector = address[9:8];
assign memWrite = (selector == 2'b00) && (writeEnable == 1'b1);
always @(posedge clk or posedge rst) begin
   if (rst)
      leds <= 16'b0;
   else if ((selector == 2'b01) && (writeLed == 1'b1))
      leds <= writeData[15:0];
end
//reading mux
```

```
always @(*) begin
   readData = 32'b0;
   if (readEnable) begin
      case (selector)
         2'b00:   readData = datamemory;
         2'b10:   readData = {16'b0, switches};
         default: readData = 32'b0;
      endcase
   end
end
endmodule
```

addressDecoderTop:

```
`timescale 1ns / 1ps
module addressDecoderTop(
   input clk, rst,
   input [31:0] address,
   input readEnable, writeEnable,
   input [31:0] writeData,
   input [15:0] switches,
   output [31:0] readData,
   output [15:0] leds
);
   wire        memWrite;
   wire [31:0] memReadData;
   Address_Decoder u_dec (
      .clk(clk),
      .rst(rst),
      .address(address),
      .readEnable(readEnable),
      .writeEnable(writeEnable),
      .writeLed(writeEnable),
      .writeData(writeData),
      .switches(switches),
      .datamemory(memReadData),
      .memWrite(memWrite),
      .readData(readData),
      .leds(leds)
   );
   DataMemory u_mem (
      .clk(clk),
      .rst(rst),
      .MemWrite(memWrite),
      .address(address),
      .write_data(writeData),
      .read_data(memReadData)
   );
endmodule
```

Switches:

```
module switches(
    input clk,
    input rst,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    output reg [31:0] readData = 0,
    output reg [15:0] leds
);
    // Emergency LED writing logic
    always @(posedge clk) begin
        if (rst)
            leds <= 16'd0;
        else if (writeEnable)
            leds <= writeData[15:0];
    end
endmodule
```

Leds:

```
module leds(
    input clk,
    input rst,
    input [15:0] btns,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    input [15:0] switches,
    output [31:0] readData
    );
    assign readData = {16'd0, switches};
endmodule
```

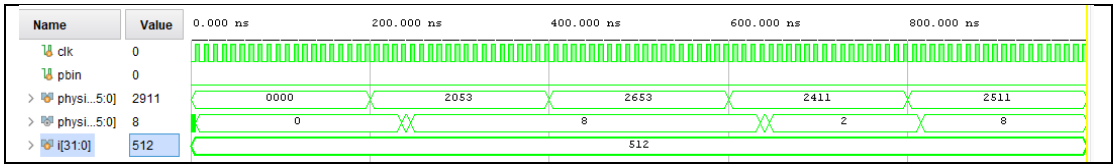
Testbench:

```
`timescale 1ns / 1ps
module memory_testbench;
    reg         clk = 0;
    reg         pbin = 0;
    reg  [15:0] physical_sw = 16'd0;
    wire [15:0] physical_leds;
    integer     i;
    top_rf_alu dut (
        .clk(clk), .pbin(pbin),
        .physical_sw(physical_sw), .physical_leds(physical_leds)
    );
    always #5 clk = ~clk;
    initial begin
```

```
        for (i = 0; i < 512; i = i + 1)
            dut.u_adt.u_mem.mem[i] = 32'd0;
        force dut.rst_clean = 1'b1;
        #100;
        force dut.rst_clean = 1'b0;
        #100;

        physical_sw = {4'b0010, 4'b0000, 4'd5, 4'd3};
        #200;
        physical_sw[11:10] = 2'b01;
        physical_sw[9]     = 1'b1;
        #200;
        physical_sw[9]   = 1'b0;
        physical_sw[3:0] = 4'd1;
        physical_sw[7:4] = 4'd1;
        #200;
        physical_sw[8] = 1'b1;
        #200;
        physical_sw[11:10] = 2'b10;
        #200;
        $finish;
    end
endmodule
```

Waveforms:



Comments: The waveform shows the memory working with the ALU and register file. With A = 3 and B = 5, the ALU gives 8, and that result is stored at address 1. After changing A and B to 1, the LEDs show 2. When readmode is turned on, the LEDs go back to 8, so the value must have come from memory and not from the ALU. At an empty address, the LEDs show 0.

Task 3

FSM

```
`timescale 1ns / 1ps
module top_rf_alu (
    input  wire       clk,
    input  wire       pbin,
    input  wire [15:0] physical_sw,
    output wire [15:0] physical_leds
);
    wire       rst_clean;
```

```
reg  [31:0] led_write_data = 32'd0;
debouncer rst_db (.clk(clk), .pbin(pbin), .pbout(rst_clean));
//LED output module
switches led_writer (
   .clk(clk), .rst(rst_clean), .writeData(led_write_data),
   .writeEnable(1'b1), .readEnable(1'b0), .memAddress(30'd0),
   .readData(), .leds(physical_leds)
);
wire [31:0] sw_read_data;
leds u_sw (
   .clk(clk),
   .rst(rst_clean),
   .btns(16'd0),
   .writeData(32'd0),
   .writeEnable(1'b0),
   .readEnable(1'b1),
   .memAddress(30'd0),
   .switches(physical_sw),
   .readData(sw_read_data)
);
//Switch mapping
wire [3:0] A_in     = sw_read_data[3:0];
wire [3:0] B_in     = sw_read_data[7:4];
wire [3:0] alu_ctrl = sw_read_data[15:12];
wire       mem_rd_mode = sw_read_data[8];
wire       mem_store   = sw_read_data[9];
wire [31:0] mem_addr    = {30'd0, sw_read_data[11:10]};
//Register file / ALU
reg        rf_we;
reg  [4:0]  rf_rd;
reg  [31:0] rf_write_data;
wire [31:0] rf_read_data1, rf_read_data2;
wire [31:0] alu_result;
wire       alu_zero;
reg state;

RegisterFile u_rf (
   .clk(clk),
   .rst(rst_clean),
   .WriteEnable(rf_we),
   .rs1(5'd1),             // read x1
   .rs2(5'd2),             // read x2
   .rd(rf_rd),
   .WriteData(rf_write_data),
   .ReadData1(rf_read_data1),
   .readData2(rf_read_data2)
);
ALU_32bit u_alu (
```

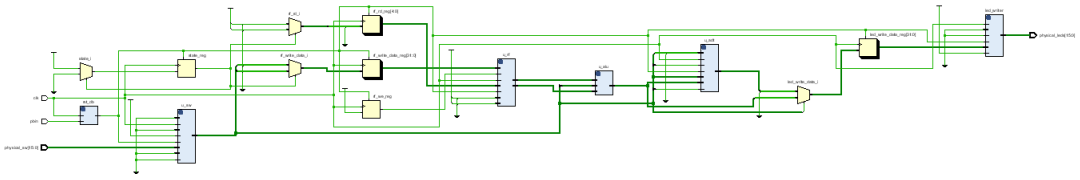
```
        .A(rf_read_data1),
        .B(rf_read_data2),
        .ALUControl(alu_ctrl),
        .ALUResult(alu_result),
        .Zero(alu_zero)
    );
    wire [31:0] dec_read_data;
    addressDecoderTop u_adt (
        .clk(clk),
        .rst(rst_clean),
        .address(mem_addr),
        .readEnable(mem_rd_mode),
        .writeEnable(mem_store),
        .writeData(alu_result),
        .switches(sw_read_data[15:0]),
        .readData(dec_read_data),
        .leds()
    );
    always @(posedge clk or posedge rst_clean) begin
        if (rst_clean) begin
            state          <= 1'b0;
            rf_we          <= 1'b0;
            rf_rd          <= 5'd0;
            rf_write_data  <= 32'd0;
            led_write_data <= 32'd0;
        end else begin
            rf_we <= 1'b1;
            case (state)
                1'b0: begin
                    rf_rd        <= 5'd1;
                    rf_write_data <= {28'd0, A_in};
                    state        <= 1'b1;
                end
                1'b1: begin
                    rf_rd        <= 5'd2;
                    rf_write_data <= {28'd0, B_in};
                    state        <= 1'b0;
                end
            endcase
            if (mem_rd_mode)
                led_write_data <= {16'd0, dec_read_data[15:0]};
            else
                led_write_data <= {16'd0, alu_result[15:0]};
        end
    end
endmodule
```

Constraints File:

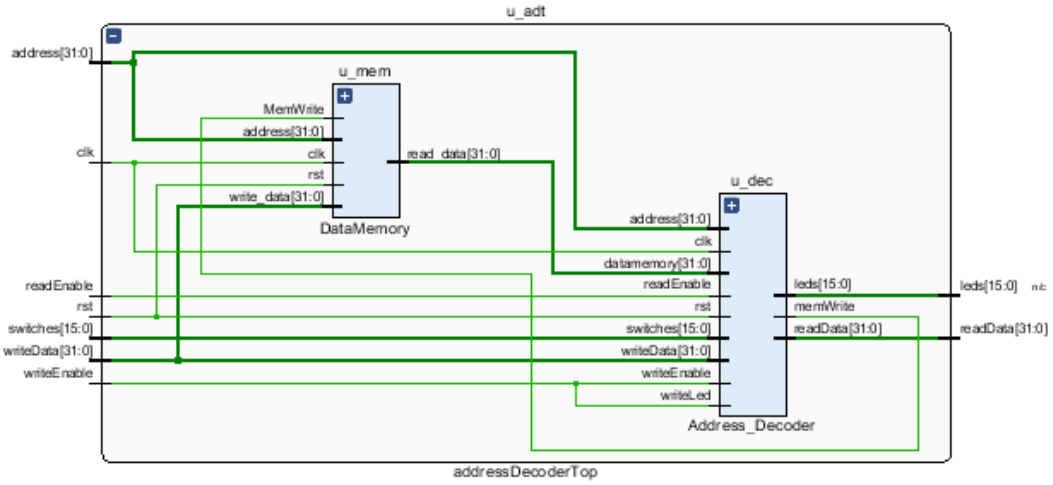
```
## Clock (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]
## Reset button (center button, BTNC)
set_property PACKAGE_PIN U18 [get_ports pbin]
set_property IOSTANDARD LVCMOS33 [get_ports pbin]
## Switches
set_property PACKAGE_PIN V17 [get_ports {physical_sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {physical_sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {physical_sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {physical_sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {physical_sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {physical_sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {physical_sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {physical_sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {physical_sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {physical_sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {physical_sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {physical_sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {physical_sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {physical_sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {physical_sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {physical_sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[*]}]
## LEDs
set_property PACKAGE_PIN U16 [get_ports {physical_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {physical_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {physical_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {physical_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {physical_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {physical_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {physical_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {physical_leds[7]}]
set_property PACKAGE_PIN V13 [get_ports {physical_leds[8]}]
set_property PACKAGE_PIN V3 [get_ports {physical_leds[9]}]
set_property PACKAGE_PIN W3 [get_ports {physical_leds[10]}]
set_property PACKAGE_PIN U3 [get_ports {physical_leds[11]}]
set_property PACKAGE_PIN P3 [get_ports {physical_leds[12]}]
set_property PACKAGE_PIN N3 [get_ports {physical_leds[13]}]
set_property PACKAGE_PIN P1 [get_ports {physical_leds[14]}]
set_property PACKAGE_PIN L1 [get_ports {physical_leds[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[*]}]
```

RTL Schematic:

Top FSM:



AddressDecoderTop:



References:

- https://github.com/achdiop/CA_Labs-Git-Abbas
- https://github.com/muhammadkabirnaqvi3-a11y/CA_LABS-Git-MuhammadKabir
- FPGA_Implementation.mp4

Lab 08 – Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.